

Rechnergestützte Numerik und Simulation

(am Beispiel von Matlab/Simulink)

Numerik 3:

Integration und Optimierung

Inhalt

3. Integration und Optimierung

- 3.1 Numerische Integration
- 3.2 Numerische Optimierung
- 3.3 Datenanalyse

3. Integration und Optimierung – 3.1 Numerische Integration

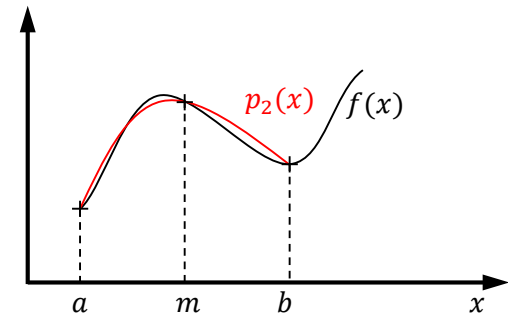
Aufgrund der zeit- und wertediskreten Charakteristik von computergestützten Simulation dynamischer Systemen, muss anstelle der analytischen Integration ein numerisches Integrationsverfahren zur Berechnung eines Integrals herangezogen werden. Die hier verwendeten Verfahren nähern die Funktion zwischen zwei Zeitpunkten auf unterschiedliche Weise an und berechnen die Fläche zwischen Approximation und Abszisse.

Die vorgestellten Befehle verwenden folgende Integrationsverfahren:

Simpsonregel

Die Simpsonregel nähert eine Funktion $f(x)$ durch eine quadratische Funktion $p_2(x)$ an, welche durch die Funktionswerte der Ursprungsfunktion $f(a)$, $f(b)$ und $f(m)$ an den Intervallgrenzen (a , b) und deren Mittelwert (m) verläuft.

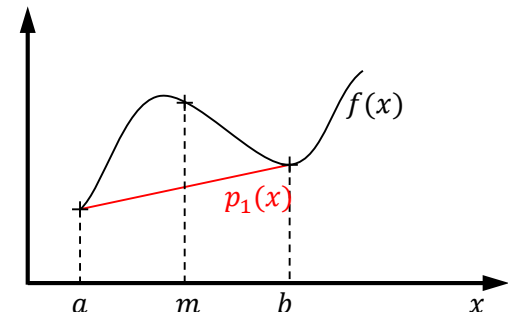
$$\int_a^b f(x) dx \approx \frac{b-a}{6} (f(a) + 4f(m) + f(b)) \text{ mit } m = \frac{a+b}{2}$$



Trapezregel

Bei der Trapezregel werden zwei Punkte durch eine gerade Verbunden und die Fläche unterhalb dieser berechnet

$$\int_a^b f(x) dx \approx \frac{b-a}{2} (f(a) + f(b))$$



3. Integration und Optimierung – 3.1 Numerische Integration

quad-Befehl:

Der Befehl `quad` integriert eine gegebene Funktion mithilfe der sogenannten adaptiven Simpson Quadratur.

Die adaptive Quadratur stellt allgemein eine rekursive Methode zur numerischen Integration dar, bei der das Integral einer Funktion in einem abgeschlossenen Intervall auf zwei verschiedene Weisen berechnet und anschließend die relative Differenz beider Ergebnisse ermittelt wird. Ist diese Differenz kleiner als eine definierter Maximalwert, wird eines der beiden Ergebnisse übernommen. Andererseits wird das Intervall in zwei Teilintervalle geteilt und der Vorgang für beide Teilfunktionen wiederholt.

Bei der adaptiven Simpson Quadratur wird die Simpson Regel erweitert und somit als Basis zur Bildung der beiden Integrationsverfahren verwendet:

```
I=quad(f,a,b,tol)
```

<code>f</code>	Funktion-handle der zu integrierende Funktion (<code>f = @sqrt</code>),
<code>a,b</code>	obere, untere Grenze des Integrationsintervalls,
<code>tol</code>	maximale relative Differenz (optional, Standard 10^{-6}),

Ist `I` ein Vektor der Form `[q n]` ist `q` das ermittelte Integral und `n` die Anzahl der berechneten Funktionswerte aus der Rekursion.

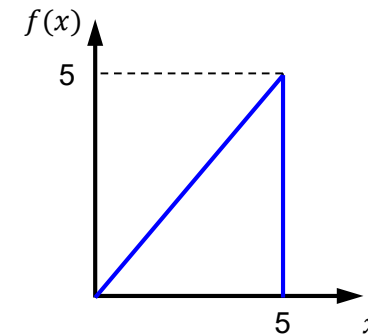
3. Integration und Optimierung – 3.1 Numerische Integration

Beispiel:

Es wird mit dem oben beschriebenen Befehl das Integral einer Dreiecksfunktion $f(x)$ in den Grenzen $[0,5]$ mit unterschiedlichen Werten für die maximale relative Differenz berechnet:

Definition einer Funktion namens `triang`

```
function y = triang(x)
if x<5
    y=x;
else
    y=0;
end
```



Verwendung der Funktion `triang` im `quad` Befehl

```
>> [q n] = quad(@triang,0,10) % tol=1e-6
```

```
q = 12.4999
n = 721
```

```
>> [q n] = quad(@triang,0,10,1e-3) % tol=1e-3
```

```
q = 12.3785
n = 241
```

3. Integration und Optimierung – 3.1 Numerische Integration

trapz-Befehl:

Die Funktion `trapz` verwendet ein Polynom ersten Grades zur Approximation des Verlaufes zwischen zwei Datenpunkten und berechnet die Fläche darunter. Im Falle dieses Befehls werden anstelle der kontinuierlichen Funktion, diskrete Datenreihen als Argumente verwendet.

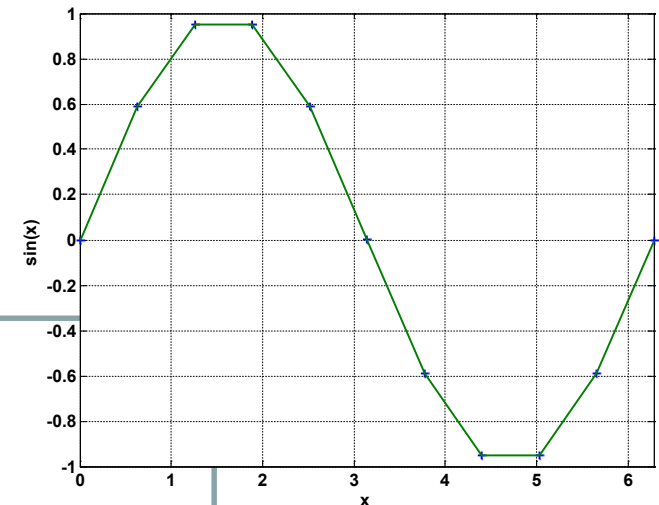
`I=trapz(x,y,)`

`x`: x-Werte der Datenpunkte

`y`: Funktionswerte der Datenpunkte

Beispiel:

```
% Anlegen der Datenpunkte  
x=0:2*pi/10:2*pi;  
y=sin(x);  
  
% Integration der mit Geraden approximierten Funktion  
I=trapz(x,y)  
  
I =  
-5.5511e-017
```



Durch die geringe Auflösung der Datenpunkte ist das berechnete Integral nicht 0.

3. Integration und Optimierung – 3.2 Numerische Optimierung

Numerische Optimierung:

MatLab Optimierungsfunktionen:

Funktion	Beschreibung
<code>fminbnd</code>	Minimierung einer Funktion mit nur einer Unbekannten in einem vorgegebenen Intervall.
<code>fminsearch</code>	Minimiert eine Funktion mit mehreren Variablen.
<code>fzero</code>	Sucht die Nullstellen einer Funktion mit nur einer Unbekannten.
<code>lsqnonneg</code>	Löst nichtnegative least-square Problemstellungen.
<code>optimget</code>	Auslesen der Optimierungsoptionen aus der Parameterstruktur.
<code>optimset</code>	Modifikation der Optimierungsoptionen der Parameterstruktur.

3. Integration und Optimierung – 3.2 Numerische Optimierung

fminsearch-Befehl:

Hinter dem Befehl `fminsearch` verbirgt sich das Nelder-Mead Simplex Verfahren [Lagarias, J. C.; Reeds, J. A; Wright, M. H.: Convergence Properties of the Nelder-Mead Simplex Method in Low Dimensions,“ *SIAM Journal of Optimization*, Vol9 Number 1, pp. 112-147, 1998.]. Das Verfahren ist aus der Übung 3.5 für eine Funktion mit nur einer Unbekannten bekannt.

```
[x,fval,exitflag,output] = fminsearch(fun,x0,options)
```

<code>x</code>	Ergebnis der Minimierung.
<code>fval</code>	Funktionswert im Minimum.
<code>exitflag</code>	gibt Auskunft über den Abbruch (1 konvergiert, 0 Maximum an Iterationen erreicht, -1 Algorithmus aus anderen Gründen abgebrochen.
<code>output</code>	gibt eine Struktur mit Informationen über die Minimierung zurück.
<code>fun</code>	Funktion die minimiert wird.
<code>x0</code>	Anfangswerte für die unbekannten Größen
<code>options</code>	Übergabe der Parameterstruktur

3. Integration und Optimierung – 3.2 Numerische Optimierung

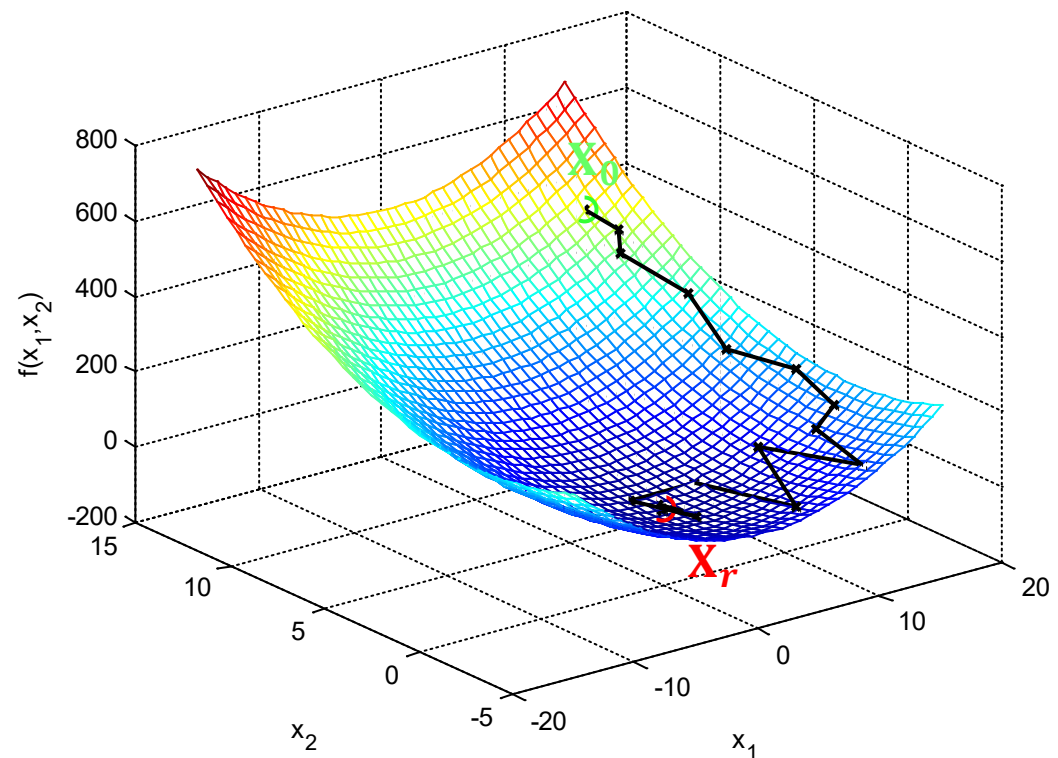
Beispiel: `fminsearch`

```
>> x0=[12 12]
fx=@(x)x(1)^2+2*x(2)^2-x(1);
options=optimset('TolX', 1e-12,'MaxIter',2000,'MaxFunEvals',2000);
[xr,fxo,flag,output]=fminsearch(fx,x0,options);
xr
```

```
x0 =
    12    12

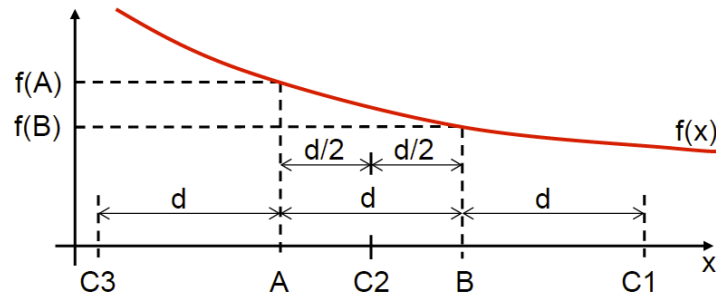
xr =
    0.5000   -0.0000

>>
```



3. Integration und Optimierung – 3.2 Numerische Optimierung

Nelder-Mead Simplex Verfahren:



1. Starte an den Stützstellen A und B. Wenn $f(A) > f(B)$, dann gehe zu $C1$ (andernfalls (*) zu $C3$).
2. Falls $f(C1) < f(A)$, (bzw. (*) $f(C3) < f(B)$), dann nehme B und $C1$ (bzw. (*) $C3$ und A) als neue Startwerte.
3. Falls $f(C1) > f(A)$, (bzw. (*) $f(C3) > f(B)$), nehme $C2$ und B (bzw. (*) A und $C2$) als neue Startwerte.

3. Integration und Optimierung – 3.2 Numerische Optimierung

`fzero`-Befehle:

Der Befehl `fzero` basiert auf eine Kombination aus Bisektion-, Sekanten- und Inverse-Quadratic-Interpolation-Verfahren [s. Literatur unter „doc fzero“]. Allgemein sucht die Funktion `fzero` eine Stelle an der das Vorzeichen wechselt.

```
[x,fval,exitflag,output] = fzero(fun,x0,options)
```

<code>x</code>	Ergebnis der Nullstellensuche.
<code>fval</code>	Funktionswert.
<code>exitflag</code>	gibt Auskunft über den Abbruch: 1 konvergiert, -1 Algorithmus aus anderen Gründen abgebrochen - s. output, -3 NaN oder Inf als Funktionswert, -4 komplexer Rückgabewert, -5 konvergierte gegen eine Singularität, -6 Vorzeichenwechsel konnte nicht gefunden werden.
<code>output</code>	gibt eine Struktur mit Informationen über die Minimierung zurück.
<code>fun</code>	Funktion bei der der Vorzeichenwechsel gesucht wird.
<code>x0</code>	Anfangswerte für die unbekannten Größen.
<code>options</code>	Übergabe der Parameterstruktur.

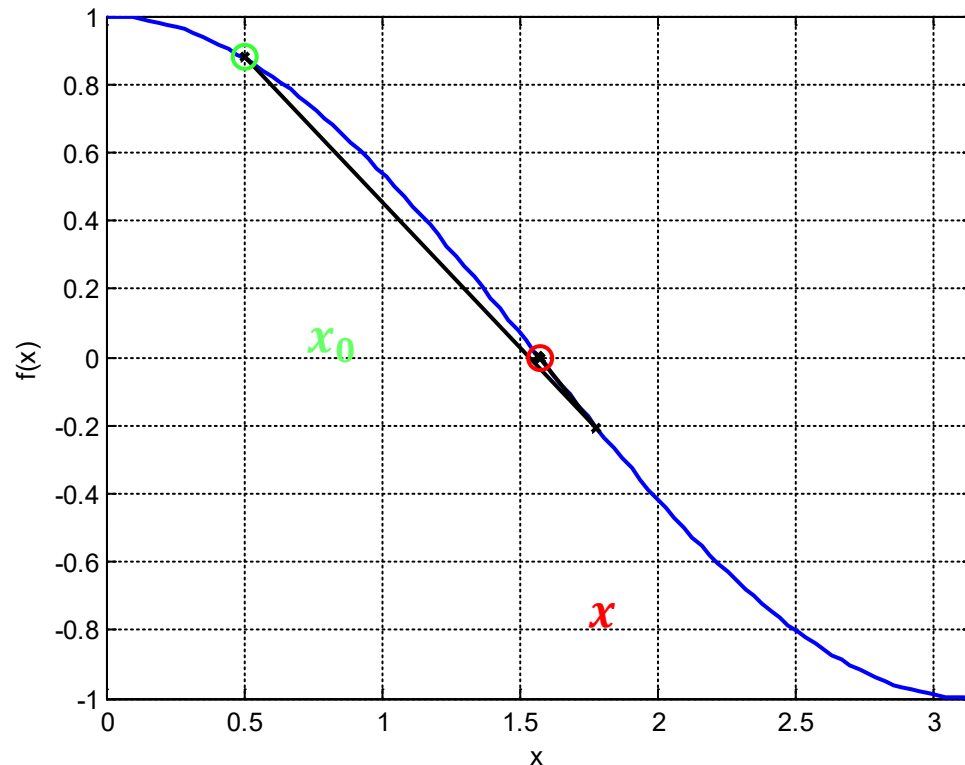
3. Integration und Optimierung – 3.2 Numerische Optimierung

Beispiel: `fzero`

```
>> x0=0.5  
    f=@(x)cos(x(1));  
    x = fzero(f,x0)
```

```
x0 =  
  
    1.5000
```

```
x =  
  
    1.5708
```



Einschränkungen:

Die Bestimmung eines Vorzeichenwechsels kommt bei einer kontinuierlichen Funktion einer Nullstelle gleich. Sind jedoch Unstetigkeiten enthalten, wie z. B. bei $\tan(x)$, wird u. U. der Wert zurückgegeben an der die Unstetigkeit auftritt.

Darüber hinaus gibt `fzero` komplexe Werte, `Inf` (Infinity) oder `NAN` (NotANumber) aus, wenn kein Nulldurchgang existiert.

3. Integration und Optimierung – 3.2 Numerische Optimierung

```
>> x0=1  
f=@(x)tan(x(1));  
[x, fval, exitflag] = fzero(f,x0)
```

x0 =

1

x =

1.5708

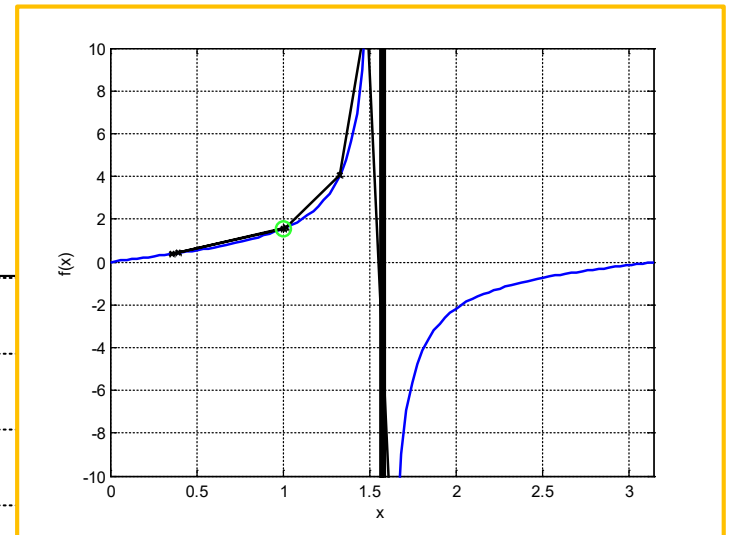
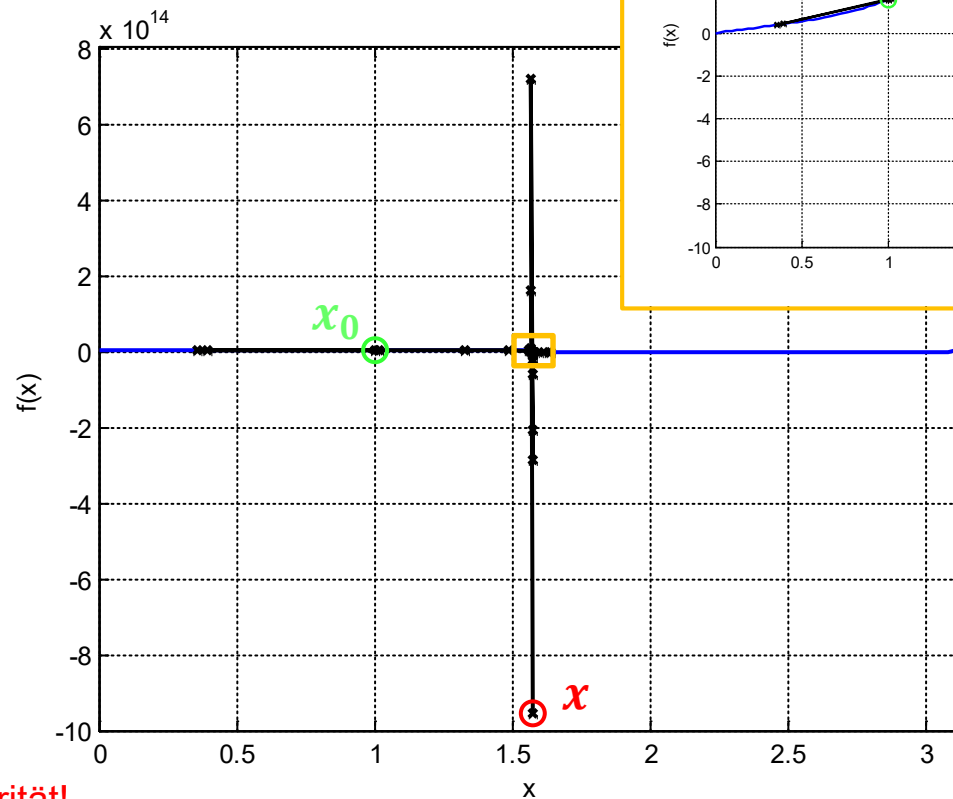
fval =

-9.5330e+014

exitflag =

-5

Singularität!



3. Integration und Optimierung – 3.3 Datenanalyse

Datenanalyse:

MALAB stellt diverse Befehle zur Datenanalyse zur Verfügung.

Übersicht:

Funktion	Beschreibung
fft	Diskrete Fourieranalyse
filter	Filterung
min / max	Minimal- und Maximalwert
mean	Mittelwert

3. Integration und Optimierung – 3.3 Datenanalyse

`fft` – **Befehle:**

Die Funktion `fft` führt auf einen Datenvektor x eine diskrete Fouriertransformation aus:

`y=fft(x,n,dim)`

<code>x</code>	– Vektor mit den x-Werten der Datenpunkte
<code>n</code>	– Stützstellenanzahl von <code>y</code>
<code>dim</code>	– Dimension

Weitere Befehle zur diskreten Fouriertransformation:

<code>fft2</code>	- 2-dim. diskrete Fouriertransformation
<code>fftn</code>	- n -dim. diskrete Fouriertransformation
<code>ifft</code>	- Inverse diskrete Fouriertransformation
<code>nextpow2</code>	- Next higher power of 2

3. Integration und Optimierung – 3.3 Datenanalyse

Definition der diskreten Fourieranalyse:

In den Funktionen $X = \text{fft}(x)$ und $x = \text{ifft}(X)$ sind folgende Umformungen implementiert:

$$X(k) = \sum_{j=1}^N x(j) d_N^{(j-1)(k-1)}$$
$$x(j) = \frac{1}{N} \sum_{k=1}^N X(k) \omega_N^{-(j-1)(k-1)}$$

mit $\omega_N = e^{-2\pi i/N}$ ist die n -te Wurzel der Einheit.

3. Integration und Optimierung – 3.3 Datenanalyse

Beispiel: `fft`-Befehle

```
TS = 1/1000;    % Abtastfrequenz
L = 1000;       % Länge
t = (0:L-1)*TS;

% 50Hz Sinusschwingung
x = sin(2*pi*50*t);
y = x + 0.2*randn(size(t));
subplot(2,1,1);
plot(t/TS,y);
title('Signal');
xlabel('Zeit in ms');
grid;

subplot(2,1,2);
NFFT = 2^nextpow2(L);
Y = fft(y,NFFT)/L;
f = 1/2/TS*linspace(0,1,NFFT/2+1);

% Amplitudenspektrum
plot(f,2*abs(Y(1:NFFT/2+1)));
title('Amplitudenspektrum');
xlabel('Frequenz in Hz');
ylabel('|Y(f)|');
```

